> A block range index entry points to a page (the atomic unit of how PostgreSQL stores data) and stores two values: the page's minimum value and the maximum value of the item to be indexed.[12]

BRIN indexes can be a good fit for analytical queries or queries that use time ranges where the data has a strong correlation between the physical ordering of the rows and how the data is queried.

Since the BRIN index stores page positions and since all rows are stored in pages, the most benefit from a BRIN index for query performance is when entire pages can be skipped.

A BRIN index only stores page values so it consumes little space. Since BRIN indexes are small, they may be worth having in addition to B-tree indexes to help for certain queries.

You saw what a block range index entry is, but what about a block range?

> A block range is a group of pages that are physically adjacent in the table

In the post *When does BRIN win?*,[13] the author shows examples of queries where the data is highly correlated, and when it has low correlation. The query uses an `AVG()` aggregate across a large data set and compares the performance with a BRIN index and a B-tree index including query times and index sizes.

In *Large Data Performance With Minimal Storage*,[14] the author again compares `AVG()` performance on large sets of data using an B-Tree Index and a BRIN index. The BRIN index not only outperforms the B-Tree Index but is massively smaller.

> "for BRIN to be effective, you need a table where the physical layout and the ordering of the column of interest are strongly correlated."

To adapt this example to Rideshare, imagine a Trip Positions model that captures GPS points from Drivers and Riders on trips. This is a high volume insert-only table where rows are not updated or deleted but might be queried.

For this example, you'll insert Trip Position records with a generated series ticking every one second over several months. This ticking second value will be used as the value for the created_at column, and will move chronologically through time from the beginning of the range to the end of the range.

12. https://www.crunchydata.com/blog/postgresql-brin-indexes-big-data-performance-with-minimal-storage
13. https://www.crunchydata.com/blog/postgres-indexing-when-does-brin-win
14. https://medium.com/geekculture/postgres-brin-index-large-data-performance-with-minimal-storage-4db6b9f64ca4